

Ex.No.1a: IMPLEMENTATION OF BASIC SEARCH STRATEGIES – 8 Puzzle

AIM:

To implement a 8-puzzle problem using Breadth First Search (BFS)

8-puzzle Problem

Definition:

- “It has set off a 3x3 board having 9 block spaces out of which 8 blocks having tiles bearing number from 1 to 8. One space is left blank. The tile adjacent to blank space can move into it. We have to arrange the tiles in a sequence for getting the goal state”.
- The puzzle can be solved by moving the tiles one by one in the single empty space and thus achieving the Goal state.

Rules of solving puzzle

- Instead of moving the tiles in the empty space we can visualize moving the empty space in place of the tile.
- The empty space can only **move in four directions** (Movement of empty space)
 1. Up
 2. Down
 3. Right or
 4. Left
- The empty space **cannot move diagonally** and can take **only one step at a time**.

Breadth-First Search

- Breadth-first search (BFS) is a traversing algorithm which starts from a selected node (source or starting node) and explores all of the neighbour nodes at the present depth before moving on to the nodes at the next level of depth.
- It must be ensured that each vertex of the graph is visited exactly once to avoid getting into an infinite loop with cyclic graphs or to prevent visiting a given node multiple times when it can be reached through more than one path.
- Breadth-first search can be implemented using a queue data structure, which follows the first-in-first-out (FIFO) method – i.e., the node that was inserted first will be visited first, and so on.

ALGORITHM:

- Step 1: We start the process by considering any random node as the starting vertex.
 - Step 2: We enqueue (insert) it to the queue and mark it as visited.
 - Step 3: Then we mark and enqueue all of its unvisited neighbours at the current depth or continue to the next depth level if there is any.
 - Step 4: The visited vertices are removed from the queue.
 - Step 5: The process ends when the queue becomes empty.
-

PROGRAM:

```
import sys

import numpy as np

class Node:

    def __init__(self, state, parent, action):

        self.state = state

        self.parent = parent

        self.action = action

class StackFrontier:

    def __init__(self):

        self.frontier = []

    def add(self, node):

        self.frontier.append(node)

    def contains_state(self, state):

        return any((node.state[0] == state[0]).all() for node in self.frontier)

    def empty(self):

        return len(self.frontier) == 0

    def remove(self):

        if self.empty():

            raise Exception("Empty Frontier")

        else:

            node = self.frontier[-1]

            self.frontier = self.frontier[:-1]

            return node

class QueueFrontier(StackFrontier):

    def remove(self):

        if self.empty():
```

```
        raise Exception("Empty Frontier")

    else:

        node = self.frontier[0]

        self.frontier = self.frontier[1:]

        return node
```

```
class Puzzle:
```

```
    def __init__(self, start, startIndex, goal, goalIndex):
```

```
        self.start = [start, startIndex]
```

```
        self.goal = [goal, goalIndex]
```

```
        self.solution = None
```

```
    def neighbors(self, state):
```

```
        mat, (row, col) = state
```

```
        results = []
```

```
        if row > 0:
```

```
            mat1 = np.copy(mat)
```

```
            mat1[row][col] = mat1[row - 1][col]
```

```
            mat1[row - 1][col] = 0
```

```
            results.append(('up', [mat1, (row - 1, col)]))
```

```
        if col > 0:
```

```
            mat1 = np.copy(mat)
```

```
            mat1[row][col] = mat1[row][col - 1]
```

```
            mat1[row][col - 1] = 0
```

```
            results.append(('left', [mat1, (row, col - 1)]))
```

```
        if row < 2:
```

```
            mat1 = np.copy(mat)
```

```
            mat1[row][col] = mat1[row + 1][col]
```

```
            mat1[row + 1][col] = 0
```

```
            results.append(('down', [mat1, (row + 1, col)]))
```

```
        if col < 2:
```

```
            mat1 = np.copy(mat)
```

```
            mat1[row][col] = mat1[row][col + 1]
```



```

        mat1[row][col + 1] = 0
        results.append(('right', [mat1, (row, col + 1)]))

    return results

def print(self):
    solution = self.solution if self.solution is not None else None
    print("Start State:\n", self.start[0], "\n")
    print("Goal State:\n", self.goal[0], "\n")
    print("\nStates Explored: ", self.num_explored, "\n")
    print("Solution:\n ")
    for action, cell in zip(solution[0], solution[1]):
        print("action: ", action, "\n", cell[0], "\n")
    print("Goal Reached!!")

def does_not_contain_state(self, state):
    for st in self.explored:
        if (st[0] == state[0]).all():
            return False
    return True

def solve(self):
    self.num_explored = 0

    start = Node(state=self.start, parent=None, action=None)
    frontier = QueueFrontier()
    frontier.add(start)

    self.explored = []

    while True:
        if frontier.empty():
            raise Exception("No solution")

        node = frontier.remove()

```

```
self.num_explored += 1
```

```
if (node.state[0] == self.goal[0]).all():  
    actions = []  
    cells = []  
    while node.parent is not None:  
        actions.append(node.action)  
        cells.append(node.state)  
        node = node.parent  
    actions.reverse()  
    cells.reverse()  
    self.solution = (actions, cells)  
    return
```

```
self.explored.append(node.state)
```

```
for action, state in self.neighbors(node.state):  
    if not frontier.contains_state(state) and self.does_not_contain_state(state):  
        child = Node(state=state, parent=node, action=action)  
        frontier.add(child)
```

```
start = np.array([[1, 2, 3], [8, 0, 4], [7, 6, 5]])
```

```
goal = np.array([[2, 8, 1], [0, 4, 3], [7, 6, 5]])
```

```
startIndex = (1, 1)
```

```
goalIndex = (1, 0)
```

```
p = Puzzle(start, startIndex, goal, goalIndex)
```

```
p.solve()
```

```
p.print()
```

OUTPUT:

Start State:

[[1 2 3]

[8 0 4]

[7 6 5]]

Goal State:

[[2 8 1]

[0 4 3]

[7 6 5]]

States Explored: 358

Solution:

action: up

[[1 0 3]

[8 2 4]

[7 6 5]]

action: left

[[0 1 3]

[8 2 4]

[7 6 5]]

action: down

[[8 1 3]

[0 2 4]

[7 6 5]]

action: right

[[8 1 3]

[2 0 4]

[7 6 5]]

action: right

[[8 1 3]

[2 4 0]

[7 6 5]]

action: up

[[8 1 0]

[2 4 3]
[7 6 5]]
action: left
[[8 0 1]
[2 4 3]
[7 6 5]]
action: left
[[0 8 1]
[2 4 3]
[7 6 5]]
action: down
[[2 8 1]
[0 4 3]
[7 6 5]]
Goal Reached!!

RESULT:

Thus the program for 8-puzzle problem using breadth-first search was implemented and executed successfully.

Ex.No:1b IMPLEMENTATION OF BASIC SEARCH STRATEGIES – 8 Queen

AIM:

To implement an 8 queens problem using Depth-first Search (DFS).

8-Queen Problem

Definition:

- The **eight queen puzzle** is the problem of placing eight chess queens on an 8×8 chessboard so that no two queens threaten each other.
- We can start the solution by putting 8 queens in 8 columns. But if you put the queens randomly there are very high chances that they will be attacking each other and hence not the solution.
- So we have to just find the row for each queen such that they don't attack among themselves.

Rules of Solving Puzzle:

- Each row can contain one and only one queen - that's pretty obvious as the queen also attacks the row therefore no other queen can be on the same row.
- Similarly we can easily see that 8 queens should occupy all 8 rows - therefore each row will have exactly one queen
- We will go from top to bottom and try to put queen on each column of the row
- When trying to put a queen we need to check if this cell is under attack or not. Instead of checking if the cell is under attack by other queens, we can check if the queen we are going to place attacks some other already placed queen or not - we can do it as attacks are symmetrical: if queen A attacks queen B this also means that queen B attacks queen A
- We need to check only 3 directions for attack:
 - i. vertical - all rows above
 - ii. diagonal - to the top left
 - iii. diagonal - to the top right

Depth-First Search:

- It is a recursive algorithm to search all the vertices of a tree data structure or a graph.
 - The depth-first search (DFS) algorithm starts with the initial node of graph G and goes deeper until we find the goal node or the node with no children.
 - Because of the recursive nature, stack data structure can be used to implement the DFS algorithm.
-

ALGORITHM:

Step 1: Place i-th queen in i-th row; only need to decide which column to use
Step 2: Start with queen 1 in row 1
Step 3: Place in column 1
Step 4: Record tentative move by pushing data onto history stack
Step 5: Move to next queen
Step 6: Try next column
Step 7: While there's conflict with any previously placed queens, move right
Step 8: If edge of board is reached, backtrack:
Step 9: pop stack, attempt to reposition that queen...

PROGRAM

```
from queue import Queue

class NQueens:

    def __init__(self, size):

        self.size = size

    def solve_dfs(self):

        if self.size < 1:

            return []

        solutions = []

        stack = [[]]

        while stack:

            solution = stack.pop()

            if self.conflict(solution):

                continue

            row = len(solution)

            if row == self.size:

                solutions.append(solution)

                continue

            for col in range(self.size):

                queen = (row, col)

                queens = solution.copy()

                queens.append(queen)

                stack.append(queens)

        return solutions

    def conflict(self, queens):
```

```

        for i in range(1, len(queens)):
            for j in range(0, i):
                a, b = queens[i]
                c, d = queens[j]
                if a == c or b == d or abs(a - c) == abs(b - d):
                    return True
            return False
        return False

def print(self, queens):
    for i in range(self.size):
        print('---' * self.size)
        for j in range(self.size):
            p = 'Q' if (i, j) in queens else ' '
            print('|' %s ' % p, end="")
        print('|')
    print('---' * self.size)

def main():
    print('.: 8-Queens Problem :.')
    size = int(input('Please enter the size of board: '))
    print_solutions = input('Do you want the solutions to be printed (Y/N): ').lower() == 'y'
    n_queens = NQueens(size)
    dfs_solutions = n_queens.solve_dfs()
    if print_solutions:
        for i, solution in enumerate(dfs_solutions):
            print('DFS Solution %d:' % (i + 1))
            n_queens.print(solution)
        print('Total DFS solutions: %d' % len(dfs_solutions))

if __name__ == '__main__':
    main()

```

OUTPUT:

∴ 8-Queens Problem ∴

Please enter the size of board: 8

Do you want the solutions to be printed (Y/N): y

DFS Solution 1:

```
-----  
| | | | | | | Q |  
-----  
| | | Q | | | | |  
-----  
| Q | | | | | | |  
-----  
| | | Q | | | | |  
-----  
| | | | | Q | | |  
-----  
| | Q | | | | | |  
-----  
| | | | | | Q | |  
-----  
| | | | Q | | | |  
-----
```

RESULT:

Thus the program for 8-queen problem using depth-first search was implemented and executed successfully.

Ex.No:1c IMPLEMENTATION OF BASIC SEARCH STRATEGIES – Cryptarithmic puzzle

AIM:

To implement an cryptarithmic puzzle using best first search.

CRYPTARITHMATIC PUZZLE:

- A cryptarithmic puzzle is a mathematical operation where the numbers are represented by letters. So each letter in the puzzle represent a certain unique digit.
- The objective is to find out the digit represented by each letter that satisfies a given equation.

RULES OF SOLVING PUZZLE:

- There should be a unique digit to be replaced with a unique alphabet.
- The result should satisfy the predefined arithmetic rules, i.e., $2+2=4$, nothing else.
- Digits should be from **0-9** only.
- There should be only one carry forward, while performing the addition operation on a problem.
- The problem can be solved from both sides, i.e., **lefthand side (L.H.S), or righthand side (R.H.S)**

BEST FIRST SEARCH:

- Best First Search is a searching algorithm which works on a set of defined rules. It makes use of the concept of priority queues and heuristic search.
- The objective of this algorithm is to reach the goal state or final state from an initial state by the shortest route possible.

ALGORITHM:

Step 1: Start by examining the rightmost digit of the topmost row, with a carry of 0

Step 2: If we are beyond the leftmost digit of the puzzle, return true if no carry, false otherwise

Step 3: If we are currently trying to assign a char in one of the addends

Step 4: If char already assigned, just recur on the row beneath this one, adding value into the sum

Step 5: If not assigned, then

- ❖ for (every possible choice among the digits not in use)
make that choice and then on row beneath this one, if successful, return true
if !successful, unmake assignment and try another digit
 - ❖ return false if no assignment worked to search
-

Step 6: Else if trying to assign a char in the sum

Step 7: If char assigned & matches correct,
recur on next column to the left with carry, if success return true,

Step 8: If char assigned & doesn't match, return false

Step 9: If char unassigned & correct digit already used, return false

Step 10: If char unassigned & correct digit unused,
assign it and recur on next column to left with carry, if success return true

Step 11: return false to trigger search

PROGRAM:

```
from queue import PriorityQueue
```

```
v = 5
```

```
graph = [[] for i in range(v)]
```

```
def best_first_search(source, target, n):
```

```
    visited = [0] * n
```

```
    visited[0] = True
```

```
    pq = PriorityQueue()
```

```
    pq.put((0, source))
```

```
    while pq.empty() == False:
```

```
        u = pq.get()[1]
```

```
        print(u, end=" ")
```

```
        if u == target:
```

```
            break
```

```
        for v, c in graph[u]:
```

```
            if visited[v] == False:
```

```
                visited[v] = True
```



```
        pq.put((c, v))

    print()

def addedge(x, y, cost):

    graph[x].append((y, cost))

    graph[y].append((x, cost))

adddedge(0, 1, 5)

adddedge(0, 2, 1)

adddedge(2, 3, 2)

adddedge(1, 4, 1)

adddedge(3, 4, 2)

source = 0

target = 4

best_first_search(source, target, v)

import itertools

def solve2():

    letters = ('s', 'e', 'n', 'd', 'm', 'o', 'r', 'y')

    digits = range(10)

    for perm in itertools.permutations(digits, len(letters)):

        sol = dict(zip(letters, perm))

        if sol['s'] == 0 or sol['m'] == 0:

            continue

        send = 1000 * sol['s'] + 100 * sol['e'] + 10 * sol['n'] + sol['d']

        more = 1000 * sol['m'] + 100 * sol['o'] + 10 * sol['r'] + sol['e']

        money = 10000 * sol['m'] + 1000 * sol['o'] + 100 * sol['n'] + 10 * sol['e'] + sol['y']
```

```
if send + more == money:

    print(" send"," more"," money")

    return send, more, money

print(solve2())
```



OUTPUT:

0 2 3 4

send more money

(9567, 1085, 10652)

RESULT:

Thus the program for cryptarithmatic problem using best-first search was implemented and executed successfully.

Ex.No:2a IMPLEMENTATION OF A* SEARCH ALGORITHM

AIM:

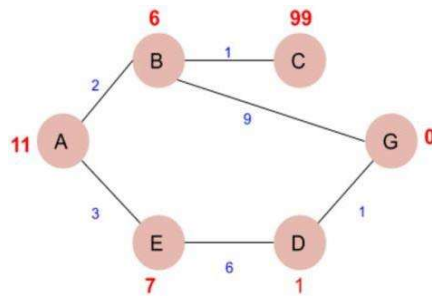
To implement a path finding using A* search algorithm.

A* SEARCH :

- A* search finds the shortest path through a search space to the goal state using the heuristic function.
- This technique finds minimal cost solutions and is directed to a goal state called A* search.
- The A* algorithm also finds the lowest-cost path between the start and goal state, where changing from one state to another requires some cost.

STEPS FOR SOLVING A* SEARCH

- Given the graph, find the cost-effective path from A to G. That is A is the source node and G is the goal node.



- Now from A, we can go to point B or E, so we compute $f(x)$ for each of them,

$$A \rightarrow B = g(B) + h(B) = 2 + 6 = 8$$

$$A \rightarrow E = g(E) + h(E) = 3 + 7 = 10$$

- Since the cost for $A \rightarrow B$ is less, we move forward with this path and compute the $f(x)$ for the children nodes of B.
- Now from B, we can go to point C or G, so we compute $f(x)$ for each of them,

$$A \rightarrow B \rightarrow C = (2 + 1) + 99 = 102$$

$$A \rightarrow B \rightarrow G = (2 + 9) + 0 = 11$$

- Here the path $A \rightarrow B \rightarrow G$ has the least cost but it is still more than the cost of $A \rightarrow E$, thus we explore this path further.
 - Now from E, we can go to point D, so we compute $f(x)$,
-

$$A \rightarrow E \rightarrow D = (3 + 6) + 1 = 10$$

- Comparing the cost of $A \rightarrow E \rightarrow D$ with all the paths we got so far and as this cost is least of all we move forward with this path.
- Now compute the $f(x)$ for the children of D

$$A \rightarrow E \rightarrow D \rightarrow G = (3 + 6 + 1) + 0 = 10$$

- Now comparing all the paths that lead us to the goal, we conclude that $A \rightarrow E \rightarrow D \rightarrow G$ is the most cost-effective path to get from A to G.

ALGORITHM:

// A* Search Algorithm

Step 1: Place the starting node into OPEN and find its $f(n)$ value.

Step 2: Remove the node from OPEN, having the smallest $f(n)$ value. If it is a goal node then stop and return success.

Step 3: Else remove the node from OPEN, find all its successors.

Step 4: Find the $f(n)$ value of all successors; place them into OPEN and place the removed node into CLOSE.

Step 5: Go to Step-2.

Step 6: Exit.

PROGRAM:

```
def aStarAlgo(start_node, stop_node):

    open_set = set(start_node)

    closed_set = set()

    g = {}          #store distance from starting node
    parents = {}    # parents contains an adjacency map of all nodes

    #distance of starting node from itself is zero
    g[start_node] = 0

    #start_node is root node i.e it has no parent nodes
    #so start_node is set to its own parent node
    parents[start_node] = start_node
```

```

while len(open_set) > 0:
    n = None

    #node with lowest f() is found
    for v in open_set:
        if n == None or g[v] + heuristic(v) < g[n] + heuristic(n):
            n = v

    if n == stop_node or Graph_nodes[n] == None:
        pass
    else:
        for (m, weight) in get_neighbors(n):
            #nodes 'm' not in first and last set are added to first
            #n is set its parent
            if m not in open_set and m not in closed_set:
                open_set.add(m)
                parents[m] = n
                g[m] = g[n] + weight

            #for each node m,compare its distance from start i.e g(m) to the
            #from start through n node
            else:
                if g[m] > g[n] + weight:
                    #update g(m)
                    g[m] = g[n] + weight
                    #change parent of m to n
                    parents[m] = n
                    #if m in closed set,remove and add to open
                    if m in closed_set:
                        closed_set.remove(m)
                        open_set.add(m)

    if n == None:
        print('Path does not exist!')
        return None

    # if the current node is the stop_node
    # then we begin reconstructin the path from it to the start_node

```

```

if n == stop_node:
    path = []
    while parents[n] != n:
        path.append(n)
        n = parents[n]
    path.append(start_node)
    path.reverse()
    print('Path found: {}'.format(path))
    return path

# remove n from the open_list, and add it to closed_list
# because all of his neighbors were inspected
open_set.remove(n)
closed_set.add(n)

print('Path does not exist!')
return None

#define fuction to return neighbor and its distance
#from the passed node
def get_neighbors(v):
    if v in Graph_nodes:
        return Graph_nodes[v]
    else:
        return None

#for simplicity we ll consider heuristic distances given
#and this function returns heuristic distance for all nodes
def heuristic(n):
    H_dist = {
        'A': 11,
        'B': 6,
        'C': 5,
        'D': 7,
        'E': 3,
        'F': 6,
        'G': 5,

```

```

        'H': 3,
        'T': 1,
        'J': 0
    }

    return H_dist[n]
#Describe your graph here
Graph_nodes = {
    'A': [('B', 6), ('F', 3)],
    'B': [('A', 6), ('C', 3), ('D', 2)],
    'C': [('B', 3), ('D', 1), ('E', 5)],
    'D': [('B', 2), ('C', 1), ('E', 8)],
    'E': [('C', 5), ('D', 8), ('I', 5), ('J', 5)],
    'F': [('A', 3), ('G', 1), ('H', 7)],
    'G': [('F', 1), ('I', 3)],
    'H': [('F', 7), ('I', 2)],
    'I': [('E', 5), ('G', 3), ('H', 2), ('J', 3)],
}

aStarAlgo('A', 'J')

```

OUTPUT:

Path found: ['A', 'F', 'G', 'I', 'J']

RESULT:

Thus the program for A* search algorithm for path was implemented and executed successfully.

Ex.No:2b IMPLEMENTATION OF MEMORY BOUNDED A* ALGORITHM

AIM:

To implement memory bounded A* search for path finding problem.

Memory bounded A* Search:

- Memory Bounded A* is **a shortest path algorithm based on the A* algorithm.**
- The main advantage is that it uses a bounded memory, while the A* algorithm might need exponential memory. All other characteristics of are inherited from A*.
- This search is an optimal and complete algorithm for finding a least-cost path. Unlike A*, it will not run out of memory, unless the size of the shortest path exceeds the amount of space in available memory.

STEPS FOR MEMORY BOUND SEARCH

Step 1: Works like A* until memory is full

Step 2: When memory is full, drop the leaf node with the highest f-value (the worst leaf), keeping track of that worst value in the parent

Step 3: Complete if any solution is reachable

Step 4: Optimal if any optimal solution is reachable

Step 5: Otherwise, returns the best reachable solution

ALGORITHM:

function SMA*(problem) return a solution sequence

 input: problem, a problem

 static: Queue, a queue of nodes ordered by f-cost

Queue $\leftarrow$ Make-Queue ({ Make-Node(INITIAL-STATE(problem)) }

Loop do

 If queue = empty return false

 N $\leftarrow$ deepest least-f cost node in queue

 If GOAL-TEST(n) return true

 S $\leftarrow$ NEXT-SUCCESSOR(n)

 if s $\neq$ goal and is maximum depth

 f(s) $\leftarrow \infty$

 else

 f(s) $\leftarrow$ Max (f(n),g(n)+h(n))

```

        if all of n's successor have been generated
            update n's f-cost and those of its ancestors if necessary
        if successors(n) all in memory then remove n from Queue
        if memory = full
            delete shallowest, highest-f-cost node in Queue
            remove it from its parent's successor list
            insert its parent on Queue if necessary
        insert s on Queue
    end

```

PROGRAM:

class Graph:

```

    def __init__(self, adjac_lis):

```

```

        self.adjac_lis = adjac_lis

```

```

        self.H1 = {

```

```

            'A': 1,

```

```

            'B': 6,

```

```

            'C': 2,

```

```

            'D': 2,

```

```

            'E': 2,

```

```

            'F': 1,

```

```

            'G': 5,

```

```

            'H': 7,

```

```

            'I': 7,

```

```

            'J': 1,

```

```

            'T': 3

```

```

        }

```

```

        self.H = {

```

```

            'A': 1,

```

```

            'B': 6,

```

```

            'C': 12,

```

'D': 10,
'E': 4,
'F': 4,
'G': 5,
'H': 7,

}

self.parent={}

self.openList=set()

self.hasRevised=[]

self.solutionGraph={}

self.solvedNodeList=set()

def get_neighbors(self, v):

 return self.adjac_lis.get(v,"")

def updateNode(self, v):

 if v in self.solvedNodeList:

 return

 feasibleChildNodeList=[]

 minimumCost=None

 minimumCostFeasibleChildNodesDict={}

 print("CURRENT PROCESSING NODE:", v)

 print("_____")

 #computing the minimum cost by visiting all child nodes with "OR/AND" condition

 for (c, weight) in self.get_neighbors(v):

```

feasibleChildNodeList=[] #initialize for computing any new feasibleChildNodes(childnodes with
minimum cost)

cost= self.getHeuristicNodeValue(c) + 1 # assuming all the edges with equal weight one
feasibleChildNodeList.append(c)
andNodesList=self.getAndNodes(v)

for nodeTuple in andNodesList: #checking whether the child(c) is in "AND" condition with other nodes
    if c in nodeTuple:
        for andNode in nodeTuple:
            if andNode!=c:
                feasibleChildNodeList.append(andNode)
                cost=cost+self.getHeuristicNodeValue(andNode) + 1 #compute total cost of "AND" nodes

if minimumCost==None: #inializing minimum cost
    minimumCost=cost
    for child in feasibleChildNodeList: #capturing parent child relationship
        self.parent[child]=v
    minimumCostFeasibleChildNodesDict[minimumCost]=feasibleChildNodeList #mapping minimum
cost child nodes

else:
    if minimumCost>cost: #checking minimum cost child nodes
        minimumCost=cost
        for child in feasibleChildNodeList:
            self.parent[child]=v
        minimumCostFeasibleChildNodesDict[minimumCost]=feasibleChildNodeList

if minimumCost==None: # no child nodes of the give node v and mark as solved node
    minimumCost=self.getHeuristicNodeValue(v)
    self.solvedNodeList.add(v)

```

```

else:

    self.setHeuristicNodeValue(v,minimumCost)      # minimum cost found! assign to the given node v
    for child in minimumCostFeasibleChildNodesDict[minimumCost]:
        if child not in self.solvedNodeList:      # checking whether minimum cost child nodes are solved
            self.openList.add(child)              # if not solved add node/s to openList for exploration
        self.solutionGraph[v]= minimumCostFeasibleChildNodesDict[minimumCost]
                                                # capture minimum cost child nodes as part of graph solution

    solved=True

    for c in self.solutionGraph.get(v,""):        # checking if feasible minimum cost child nodes are already
solved
        if c not in self.solvedNodeList:
            solved=solved & False

    if solved == True:                          # if all the feasible child nodes of the given node are solved
        self.solvedNodeList.add(v)              # mark the given node v as solved


print("HEURISTIC VALUES :", self.H)
print("OPEN LIST      :", list(self.openList))
print("MINIMUM COST NODES:", minimumCostFeasibleChildNodesDict.get(minimumCost,"[ ]"))
print("SOLVED NODE LIST :", list(self.solvedNodeList))
print("-----")

def getAndNodes(self,v):
    andNodes={
        # PARENT NODE AS KEY FOR ITS CHILD NODES IN AND CONDITION
        'A':[('B','C')],
        'D':[('E','F')]
    }

    return andNodes.get(v, "")

```

```

def getHeuristicNodeValue(self, n):
    return self.H.get(n,0) #Always return the heuristic value

def setHeuristicNodeValue(self, n, value):
    self.H[n]=value      # sets the revised heuristic value of a give node

def ao_star_algorithm(self, start):
    # In this open_list is a list of nodes which have been visited, but who's
    # neighbours haven't all been always inspected, It starts off with the start node.
    # And closedList is a list of nodes which have been visited and who's neighbors have been always
    inspected.
    self.openList = set([start])

    while len(self.openList) > 0:

        # it will find a node with the lowest value of f() -
        v = self.openList.pop()      # procesess a Node in the openList

        self.updateNode(v)

        while v!=start and self.parent[v] not in self.solvedNodeList:

            parent=self.parent[v];
            self.updateNode(parent)
            v=parent

        print("TRAVERSE SOLUTION FROM ROOT TO COMPUTE THE FINAL SOLUTION GRAPH")
        print("-----")
        print("SOLUTION GRAPH:",self.solutionGraph)
        print("\n")

nodeList1 = {
    'A': [('B', 1), ('C', 1), ('D', 1)],

```

```
{
    'B': [('G', 1), ('H', 1)],
    'C': [('J', 1)],
    'D': [('E', 1), ('F', 1)],
    'G': [('I', 1)]

}

nodeList = {
    'A': [('B', 1), ('C', 1), ('D', 1)],
    'B': [('G', 1), ('H', 1)],
    'D': [('E', 1), ('F', 1)]

}

graph = Graph(nodeList)
graph.ao_star_algorithm('A')
```

OUTPUT:

CURRENT PROCESSING NODE: A

HEURISTIC VALUES : {'A': 11, 'B': 6, 'C': 12, 'D': 10, 'E': 4, 'F': 4, 'G': 5, 'H': 7}

OPEN LIST : ['D']

MINIMUM COST NODES: ['D']

SOLVED NODE LIST : []

TRAVERSE SOLUTION FROM ROOT TO COMPUTE THE FINAL SOLUTION GRAPH

SOLUTION GRAPH: {'A': ['D']}

CURRENT PROCESSING NODE: D

HEURISTIC VALUES : {'A': 11, 'B': 6, 'C': 12, 'D': 10, 'E': 4, 'F': 4, 'G': 5, 'H': 7}

OPEN LIST : ['F', 'E']

MINIMUM COST NODES: ['E', 'F']

SOLVED NODE LIST : []

CURRENT PROCESSING NODE: A

HEURISTIC VALUES : {'A': 11, 'B': 6, 'C': 12, 'D': 10, 'E': 4, 'F': 4, 'G': 5, 'H': 7}

OPEN LIST : ['F', 'E', 'D']

MINIMUM COST NODES: ['D']

SOLVED NODE LIST : []

TRAVERSE SOLUTION FROM ROOT TO COMPUTE THE FINAL SOLUTION GRAPH

SOLUTION GRAPH: {'A': ['D'], 'D': ['E', 'F']}

CURRENT PROCESSING NODE: F

HEURISTIC VALUES : {'A': 11, 'B': 6, 'C': 12, 'D': 10, 'E': 4, 'F': 4, 'G': 5, 'H': 7}

OPEN LIST : ['E', 'D']

MINIMUM COST NODES: []

SOLVED NODE LIST : ['F']

CURRENT PROCESSING NODE: D

HEURISTIC VALUES : {'A': 11, 'B': 6, 'C': 12, 'D': 10, 'E': 4, 'F': 4, 'G': 5, 'H': 7}

OPEN LIST : ['E', 'D']

MINIMUM COST NODES: ['E', 'F']

SOLVED NODE LIST : ['F']

CURRENT PROCESSING NODE: A

HEURISTIC VALUES : {'A': 11, 'B': 6, 'C': 12, 'D': 10, 'E': 4, 'F': 4, 'G': 5, 'H': 7}

OPEN LIST : ['E', 'D']

MINIMUM COST NODES: ['D']

SOLVED NODE LIST : ['F']

TRAVERSE SOLUTION FROM ROOT TO COMPUTE THE FINAL SOLUTION GRAPH

SOLUTION GRAPH: {'A': ['D'], 'D': ['E', 'F']}

CURRENT PROCESSING NODE: E

HEURISTIC VALUES : {'A': 11, 'B': 6, 'C': 12, 'D': 10, 'E': 4, 'F': 4, 'G': 5, 'H': 7}

OPEN LIST : ['D']

MINIMUM COST NODES: []

SOLVED NODE LIST : ['F', 'E']

CURRENT PROCESSING NODE: D

HEURISTIC VALUES : {'A': 11, 'B': 6, 'C': 12, 'D': 10, 'E': 4, 'F': 4, 'G': 5, 'H': 7}

OPEN LIST : ['D']

MINIMUM COST NODES: ['E', 'F']

SOLVED NODE LIST : ['F', 'D', 'E']

CURRENT PROCESSING NODE: A

HEURISTIC VALUES : {'A': 11, 'B': 6, 'C': 12, 'D': 10, 'E': 4, 'F': 4, 'G': 5, 'H': 7}

OPEN LIST : ['D']

MINIMUM COST NODES: ['D']

SOLVED NODE LIST : ['F', 'D', 'A', 'E']

TRAVERSE SOLUTION FROM ROOT TO COMPUTE THE FINAL SOLUTION GRAPH

SOLUTION GRAPH: {'A': ['D'], 'D': ['E', 'F']}

TRAVERSE SOLUTION FROM ROOT TO COMPUTE THE FINAL SOLUTION GRAPH

SOLUTION GRAPH: {'A': ['D'], 'D': ['E', 'F']}

RESULT:

Thus the program for memory bounded A* search was implemented and executed successfully.



Ex.No:3 IMPLEMENTATION OF MINIMAX ALGORITHM FOR GAME PLAYING (ALPHA –BETA PRUNING)

AIM:

To implement a Tic-Tac Toe program using Minmax Alpha Beta Pruning algorithm.

MINIMAX ALGORITHM:

- Minimax is a kind of backtracking algorithm that is used in decision making and game theory to find the optimal move for a player, assuming that your opponent also plays optimally.
- Alpha–beta pruning is **a search algorithm that seeks to decrease the number of nodes that are evaluated by the minimax algorithm in its search tree.**

ALGORITHM:

function minimax(node, depth, isMaximizingPlayer, alpha, beta):

if node is a leaf node :
 return value of the node

if isMaximizingPlayer :
 bestVal = -INFINITY
 for each child node :
 value = minimax(node, depth+1, false, alpha, beta)
 bestVal = max(bestVal, value)
 alpha = max(alpha, bestVal)
 if beta <= alpha:
 break
 return bestVal

else :
 bestVal = +INFINITY
 for each child node :
 value = minimax(node, depth+1, true, alpha, beta)
 bestVal = min(bestVal, value)
 beta = min(beta, bestVal)
 if beta <= alpha:
 break
 return bestVal

PROGRAM:

```
from functools import lru_cache
from copy import copy
from typing import Tuple, List, Iterable, Union
from multiprocessing import Pool
from math import inf, ceil

class players():
    X = -1
    O = 1

class Board(list):
    __all__ = ('check', 'ai_move', 'empty_cells', 'cell_for_check', 'score2status', 'move', 'is_win',
              'is_tie', 'is_end', 'clear', 'copy')
    def __init__(self, size: int=3):
        self.size=size
        super().__init__([None]*size*size)

    @staticmethod
    @lru_cache
    def cell_for_check(n) -> Tuple[Tuple[int]]:
        reuslt = []
        reuslt.extend(tuple(i+ii-1 for i in range(1, n*n+1, n)) for ii in range(n)) # -
        reuslt.extend(tuple(i+ii-1 for i in range(n)) for ii in range(1, n*n+1, n)) # |
        reuslt.append(tuple(i*n - (n-i)-1 for i in range(1, n+1))) # \
        reuslt.append(tuple(i*n - (i-1)-1 for i in range(1, n+1))) # /
        return tuple(reuslt)

    def empty_cells(self) -> Iterable[int]:
        return [i for i in range(self.size*self.size) if self[i] == None]
```

```

def ai_move(self, player: int, max_depth: Union[int, None]=None, multiprocessing:
bool=True) -> Tuple[int, Union[int, float], int]:
    """Move the player with ai, returns move, score, depth"""
    assert not self.is_end(), 'Game is End'
    best_score = -inf
    best_move = None
    max_depth = max_depth if max_depth != None else self.size*self.size

    if self.count(None) == self.size*self.size:
        # if start first, use central cell
        best_move = ceil(self.size/2) * self.size - (self.size - ceil(self.size/2)) - 1
        self[best_move] = player
        return best_move, inf, self.size*self.size

    if multiprocessing:
        with Pool() as p:
            result = p.starmap(
                Board.alpha_beta,
                ((self.copy().move(i+1, player), player, max_depth,) for i in self.empty_cells()),
                )
    else:
        result = map(
            Board.alpha_beta,
            [self.copy().move(i+1, player) for i in self.empty_cells()],
            (player,)*len(self.empty_cells()),
            (max_depth,)*len(self.empty_cells()),
            )

    for key, (score, depth,) in zip(self.empty_cells(), result):
        if score > best_score:
            best_score = score

```

```

        best_move = key
    if score == 1: # if is win move, break the loop
        break

    self[best_move] = player
    return best_move, best_score, max_depth-depth+1

def move(self, key: int, player: int):
    key -= 1
    assert self[key] == None, 'Bad cell'
    self[key] = player
    return self

def alpha_beta(
    self,
    player: int,
    depth: int,
    alpha: Union[int, float] = -inf,
    beta: Union[int, float] = inf,
    is_max: bool = False
) -> Tuple[int, int]:
    if self.is_win(player): # win
        return 1, depth
    elif self.is_win(-player): # lose
        return -1, depth
    elif self.is_tie() or depth <= 0: # tie
        return 0, depth

    if is_max:

```

```

best_score = -inf
for key in self.empty_cells():
    self[key] = player
    score, d = self.alpha_beta(player, depth-1, alpha, beta, False)
    self[key] = None
    if score > best_score:
        best_score = score
    if score > alpha:
        alpha = score
    if beta <= alpha:
        break
return best_score, d
else:
    worst_score = inf
    for key in self.empty_cells():
        self[key] = -player
        score, d = self.alpha_beta(player, depth-1, alpha, beta, True)
        self[key] = None
        if score < worst_score:
            worst_score = score
        if score < beta:
            beta = score
        if beta <= alpha:
            break
    return worst_score, d

```

@staticmethod

def score2status(score: Union[int, float]) -> str:

```

return (
    'tie' if score == 0 else

```

```
        'win' if score > 0 else
        'lose'
    )
```

```
def is_win(self, player: int) -> bool:
    for row in self.cell_for_check(self.size):
        if all(self[i] == player for i in row):
            return True
    return False
```

```
is_tie: bool = lambda self: self.count(None) == 0
```

```
is_end: bool = lambda self: self.is_tie() or self.is_win(players.O) or self.is_win(players.X)
```

```
def clear(self):
    """Clear the board."""
    super().__init__([None]*self.size*self.size)
```

```
def copy(self):
    """Return a shallow copy of the board."""
    return copy(self)
```

```
def __repr__(self) -> str:
    PLAYERS = {players.X: 'X', players.O: 'O', None: ' '}
    ROW = f+{"-"*3}' * self.size + '+\n'
    string = ROW
    for i in range(0, self.size*self.size, self.size):
        string += '|' + '|'.join(f'{PLAYERS[i:^3}]' for i in self[i: i+self.size]) + f'\n{ROW}'
    return string[:-1]
```

```
if __name__ == '__main__':
```

```

from time import time
board = Board(3)
last_player = players.X
while 1:
    t1 = time()
    move, score, depth = board.ai_move(last_player, multiprocess=False)
    t2 = time()
    tT = round(t2-t1, 3)
    print(
        (
            f'processed depth: {depth}, '
            f'status: {board.score2status(score)}, '
            f'move: {move+1}, '
            f'{tT}s\n'
        ),
        board
    )

    last_player = -last_player # convert players
    if board.is_end():
        print(
            'Game is end,',
            'Tie' if board.is_tie() else
            'X Win' if board.is_win(players.X) else
            'O Win'
        )
        #board.clear()
        Break

```

OUTPUT:

processed depth: 9, status: win, move: 5, 0.0s

```
+---+---+---+
|   |   |   |
+---+---+---+
|  |X|  |  |
+---+---+---+
|   |   |   |
+---+---+---+
```

processed depth: 7, status: tie, move: 1, 0.083s

```
+---+---+---+
|O|  |  |  |
+---+---+---+
|  |X|  |  |
+---+---+---+
|   |   |   |
+---+---+---+
```

processed depth: 7, status: tie, move: 2, 0.008s

```
+---+---+---+
|O|X|  |  |
+---+---+---+
|  |X|  |  |
+---+---+---+
|   |   |   |
+---+---+---+
```

processed depth: 2, status: tie, move: 8, 0.0s

```
+---+---+---+
|O|X|  |  |
+---+---+---+
|  |X|  |  |
+---+---+---+
|  |O|  |  |
```

+---+---+---+

processed depth: 5, status: tie, move: 4, 0.0s

+---+---+---+

| O | X | |

+---+---+---+

| X | X | |

+---+---+---+

| | O | |

+---+---+---+

processed depth: 4, status: tie, move: 6, 0.016s

+---+---+---+

| O | X | |

+---+---+---+

| X | X | O |

+---+---+---+

| | O | |

+---+---+---+

processed depth: 3, status: tie, move: 3, 0.0s

+---+---+---+

| O | X | X |

+---+---+---+

| X | X | O |

+---+---+---+

| | O | |

+---+---+---+

processed depth: 2, status: tie, move: 7, 0.0s

+---+---+---+

| O | X | X |

+---+---+---+

| X | X | O |

+---+---+---+

| O | O | |

+---+---+---+

processed depth: 1, status: tie, move: 9, 0.0s

+---+---+---+

| O | X | X |

+---+---+---+

| X | X | O |

+---+---+---+

| O | O | X |

+---+---+---+

Game is end, Tie

RESULT:

Thus the game playing program using Minmax algorithm was implemented and executed successfully.

Ex.No:4 IMPLEMENTATION OF CONSTRAINT SATISFACTION PROBLEMS

AIM:

To implement a map colouring program with constraint satisfaction problem.

CONSTRAINT SATISFACTION PROBLEM:

- Constraint satisfaction is a technique where a problem is solved when its values satisfy certain constraints or rules of the problem.
- Constraint satisfaction depends on three components, namely:
 - i.* **X:** It is a set of variables.
 - ii.* **D:** It is a set of domains where the variables reside. There is a specific domain for each variable.
 - iii.* **C:** It is a set of constraints which are followed by the set of variables.
- The map-coloring CSP requires that you assign a color to each region of a map such that any two regions sharing a border have different colors.

PROGRAM:

R is a set of restrictions

this functions colors the given province with the given color

returns false if not possible, returns the set of new restrictions if possible

def addColor(R, province, color):

 ans = []

 for rr in R:

 res = checkRestriction(rr, province, color)

 if res == False:

 return False

 elif res == None:

 continue

 else:

 ans.append(res)

 return ans

checks if the restrition rr allows the given province to have the given color

returns false if not possible, otherwise returns the new restriction

```

def checkRestriction(rr, province, color):
    #finding the index of the province (saved to index)
    index = -1
    other = -1
    if rr[0] == province:
        index = 0
        other = 1
    elif rr[1] == province:
        index = 1
        other = 0
    else:
        return rr

    if isinstance(rr[other], int):
        # other component is a color
        if (color != rr[other]):
            return None
        else:
            return False
    else:
        return [rr[other], color]

# solving the CSP by variable elimination
# recursive structure: ci is the province index to be colored (0 = bc, 1 = ab, etc)
# n is the number of colors
# provinces is a list of provinces
# if coloring is possible returns the province-> color map, otherwise False
def solveCSP(provinces, n, R, ci):
    if (ci == 0):

```

```

# in the beginning any color can be assigned to the first province, lets say 1
newR = addColor(R, provinces[0], 1)
if(newR == False):
    return False
ans = {provinces[0]:1}
res = solveCSP(provinces, n, newR, 1)
if(res == False):
    return False
ans.update(res)
return ans
elif(ci == len(provinces)):
    return {}

# branching over all possible colors for provinces[ci]
for color in range (1,n+1):
    ans = {provinces[ci]:color}
    newR = addColor(R, provinces[ci], color)
    if(newR == False):
        continue
    res = solveCSP(provinces, n, newR, ci+1)
    if(res == False):
        continue
    #print(ans)
    #print(res)
    #print("=====")
    ans.update(res)
    return ans

# no choice for the current province
return False

```

```

# main program starts

# =====

n=5 #int(input("Enter the number of color"))
colors=[]
for i in range(1,n+1):
    colors.append(i)
#print(colors)

# creating map of canada
# cmap[x] gives the neighbors of the province x
cmap = {}
cmap["ab"] = ["bc","nt","sk"]
cmap["bc"] = ["yt", "nt", "ab"]
cmap["mb"] = ["sk","nu","on"]
cmap["nb"] = ["qc", "ns", "pe"]
cmap["ns"] = ["nb", "pe"]
cmap["nl"] = ["qc"]
cmap["nt"] = ["bc", "yt", "ab", "sk", "nu"]
cmap["nu"] = ["nt", "mb"]
cmap["on"] = ["mb", "qc"]
cmap["pe"] = ["nb", "ns"]
cmap["qc"] = ["on", "nb", "nl"]
cmap["sk"] = ["ab", "mb", "nt"]
cmap["yt"] = ["bc", "nt"]

# CSP restrictions
# each restriction is modeled as a pair [a,b] which means the province a's
# color is not equal to b, where b is either a color (a number 1 to n) or
# another province. Examples ['bc', 'ab'] means the color of bc should

```

not be equal to ab -- ["bc",4] means the color of bc should not be 4

R is the list of restrictions

R = []

initiating restrictions based on the province neighborhood

for x in cmap:

 for y in cmap[x]:

 R.append([x,y])

initiating a list of provinces

provinces = []

for p in cmap:

 provinces.append(p)

#print(solveCSP(provinces, 3, R, 0))

while(1):

 num=int(input("Enter number of the color? "))

 print(solveCSP(provinces, num, R, 0))

#print(R)

#print(" ===== ")

#print(checkRestriction(["ab",4], "ab", 4))

#R = addColor(R, 'bc', 4)

#print(R)

#print(" ===== ")

#print(checkRestriction(["ab",4], "ab", 4))

#R = addColor(R, 'ab', 4)

#print(R)

OUTPUT:

Enter number of the color? 3

```
{'ab': 1, 'bc': 2, 'mb': 1, 'nb': 1, 'ns': 2, 'nl': 1, 'nt': 3, 'nu': 2, 'on': 2, 'pe': 3, 'qc': 3, 'sk': 2, 'yt': 1}
```

RESULT:

Thus the map coloring program using constraint satisfaction was implemented and executed successfully.

Ex.No:5 IMPLEMENTATION OF PROPOSITIONAL MODEL CHECKING ALGORITHMS

AIM:

To implement a minesweeper puzzle game using propositional logic.

PROPOSITIONAL LOGIC ALGORITHM:

- Representing knowledge is the key issue in Artificial Intelligence. We can develop a knowledgeable AI agent that can analyze like humans.
- The goal in this project will be to build an AI that can play Minesweeper. Recall that knowledge-based agents make decisions by considering their knowledge base, and making inferences based on that knowledge.
- One way we could represent an AI's knowledge about a Minesweeper game is by making each cell a propositional variable that is true if the cell contains a mine, and false otherwise.

STEPS FOR MINESWEEPER PUZZLE:

Step 1: The player is presented with a rectangular grid of tiles, behind which are hidden a certain number of randomly distributed mines.

Step 2: The player uncovers a tile by clicking it; if the clicked square is not a mine, it reveals an integer, which is the number of adjacent uncovered tiles that contain a mine.

Step 3: Given new information, the player need to decide which square to uncover next until the uncovered square is a mine in an attempt or the board is cleared.

PROGRAM:

```
import random

def GenerateMineSweeperMap(n, k):
    arr = [[0 for row in range(n)] for column in range(n)]
    for num in range(k):
        x = random.randint(0,n-1)
        y = random.randint(0,n-1)
        arr[y][x] = 'X'
        if (x >=0 and x <= n-2) and (y >= 0 and y <= n-1):
            if arr[y][x+1] != 'X':
                arr[y][x+1] += 1 # center right
        if (x >=1 and x <= n-1) and (y >= 0 and y <= n-1):
```

```

        if arr[y][x-1] != 'X':
            arr[y][x-1] += 1 # center left
    if (x >= 1 and x <= n-1) and (y >= 1 and y <= n-1):
        if arr[y-1][x-1] != 'X':
            arr[y-1][x-1] += 1 # top left

    if (x >= 0 and x <= n-2) and (y >= 1 and y <= n-1):
        if arr[y-1][x+1] != 'X':
            arr[y-1][x+1] += 1 # top right
    if (x >= 0 and x <= n-1) and (y >= 1 and y <= n-1):
        if arr[y-1][x] != 'X':
            arr[y-1][x] += 1 # top center

    if (x >= 0 and x <= n-2) and (y >= 0 and y <= n-2):
        if arr[y+1][x+1] != 'X':
            arr[y+1][x+1] += 1 # bottom right
    if (x >= 1 and x <= n-1) and (y >= 0 and y <= n-2):
        if arr[y+1][x-1] != 'X':
            arr[y+1][x-1] += 1 # bottom left
    if (x >= 0 and x <= n-1) and (y >= 0 and y <= n-2):
        if arr[y+1][x] != 'X':
            arr[y+1][x] += 1 # bottom center

    return arr

def GeneratePlayerMap(n):
    arr = [['-' for row in range(n)] for column in range(n)]
    return arr

def DisplayMap(map):
    for row in map:
        print(" ".join(str(cell) for cell in row))
    print("")

```

```

def CheckWon(map):
    for row in map:
        for cell in row:
            if cell == '-':
                return False
    return True

def CheckContinueGame(score):
    print("Your score: ", score)
    isContinue = input("Do you want to try again? (y/n) :")
    if isContinue == 'n':
        return False
    return True

def Game():
    GameStatus = True
    while GameStatus:
        difficulty = input("Select your difficulty (b, i, h):")
        if difficulty.lower() == 'b':
            n = 5
            k = 3
        elif difficulty.lower() == 'i':
            n = 6
            k = 8
        else:
            n = 8
            k = 20

        minesweeper_map = GenerateMineSweeperMap(n, k)
        player_map = GeneratePlayerMap(n)
        score = 0
        while True:

```

```

if CheckWon(player_map) == False:
    print("Enter your cell you want to open :")
    x = input("X (1 to 5) :")
    y = input("Y (1 to 5) :")
    x = int(x) - 1 # 0 based indexing
    y = int(y) - 1 # 0 based indexing
    if (minesweeper_map[y][x] == 'X'):
        print("Game Over!")
        DisplayMap(minesweeper_map)
        GameStatus = CheckContinueGame(score)
        break
    else:
        player_map[y][x] = minesweeper_map[y][x]
        DisplayMap(player_map)
        score += 1

else:
    DisplayMap(player_map)
    print("You have Won!")
    GameStatus = CheckContinueGame(score)
    break

# Start of Program
if __name__ == "__main__":
    try:
        Game()
    except KeyboardInterrupt:
        print('\nEnd of Game. Bye Bye!')

```

OUTPUT:

Select your difficulty (b, i, h):i

Enter your cell you want to open :

X (1 to 5) :3

Y (1 to 5) :4

Game Over!

1 X 2 1 1 0

1 1 3 X 2 0

0 1 3 X 3 1

0 1 X 4 X 2

0 2 2 5 X 3

0 1 X 3 X 2

Your score: 0

Do you want to try again? (y/n) :

RESULT:

Thus the program for knowledge based minesweeper puzzle game with propositional model checking was implemented and executed successfully.

Ex.No:6a IMPLEMENTATION OF FORWARD CHAINING

AIM:

To implement a program using forward chaining strategy

FORWARD CHAINING:

- Forward chaining starts from known facts and applies inference rule to extract more data until it reaches to the goal.
- As it moves from bottom to top, it is a bottom-up method.
- Forward chaining can generate an infinite number of possible conclusions.
- It operates in the forward direction.
- It is a method of arriving at a conclusion based on known facts or data by starting at the beginning and working one's way to the end.

STEPS IN FORWARD CHAINING:

- A simple example of forward chaining can be explained in the following sequence.

A

A->B

B

- Where, A is the starting point. A->B represents a fact. This fact is used to achieve a decision B.
- A practical example will go as follows;

Tom is running (A)

If a person is running, he will sweat (A->B)

Therefore, Tom is sweating. (B)

PROGRAM:

global facts

global is_changed

is_changed = True

facts = [["vertebrate","duck"],["flying","duck"],["mammal","cat"]]

def assert_fact(fact):

 global facts

 global is_changed

 if not fact in facts:

 facts += [fact]

 is_changed = True

while is_changed:

 is_changed = False

 for A1 in facts:

 if A1[0] == "mammal":

 assert_fact(["vertebrate",A1[1]])

 if A1[0] == "vertebrate":

 assert_fact(["animal",A1[1]])

 if A1[0] == "vertebrate" and ["flying",A1[1]] in facts:

 assert_fact(["bird",A1[1]])

print(facts)

OUTPUT:

```
['vertebrate', 'duck'], ['flying', 'duck'], ['mammal', 'cat'], ['animal', 'duck'], ['bird', 'duck'],  
['vertebrate', 'cat'], ['animal', 'cat']]
```

RESULT:

Thus the program for forward chaining strategy was implemented and executed successfully.

Ex.No:6b IMPLEMENTATION OF BACKWARD CHAINING

AIM:

To implement a program for backward chaining strategy

BACKWARD CHAINING:

- Backward chaining is an idea in AI that includes backtracking from the endpoint or objective to steps that prompted the endpoint.
- Backward Chaining is a top-down approach where we start from the goal state and works backwards to find the required facts that support the goal statement.
- It applies the Depth-First search strategy. It can only generate a limited number of conclusions. It only tests for few of the required rules

STEPS IN BACKWARD CHAINING:

- Backward chaining can be explained in the following sequence.

B

A->B

A

- Where, B is the goal or endpoint, that is used as the starting point for backward tracking. A is the initial state. A->B is a fact that must be asserted to arrive at the endpoint B.
- A practical example of backward chaining will go as follows:

Tom is sweating (B).

If a person is running, he will sweat (A->B).

Tom is running (A).

PROGRAM:

global facts

global is_changed

is_changed = True

facts = [["plant","mango"],["eating","mango"],["seed","sprouts"]]

def assert_fact(fact):

 global facts

 global is_changed

 if not fact in facts:

 facts += [fact]

 is_changed = True

while is_changed:

 is_changed = False

 for A1 in facts:

 if A1[0] == "seed":

 assert_fact(["plant",A1[1]])

 if A1[0] == "plant":

 assert_fact(["fruit",A1[1]])

 if A1[0] == "plant" and ["eating",A1[1]] in facts:

 assert_fact(["human",A1[1]])

print(facts)

OUTPUT:

[['plant', 'mango'], ['eating', 'mango'], ['seed', 'sprouts'], ['fruit', 'mango'], ['human', 'mango'],
['plant', 'sprouts'], ['fruit', 'sprouts']]

RESULT:

Thus the program for forward chaining strategy was implemented and executed successfully.

Ex.No:6c IMPLEMENTATION OF RESOLUTION STRATEGY

AIM:

To implement a program for inference using Resolution.

RESOLUTION STRATEGY

- Resolution is a method of inference leading to a refutation theorem-proving technique for sentences in propositional logic and first-order logic(FOL).
- In other words, iteratively applying the resolution rule along with unification in a suitable way allows for inferencing whether a CNF statement is satisfiable or unsatisfiable.
- Attempting to prove a satisfiable FOL statement as unsatisfiable may result in a nonterminating computation; this problem doesn't occur in propositional logic. Resolution works on statements in Conjunctive Normal Form(CNF).
- Resolution is a complete and sound inference procedure because it works on CNF which is universal.

STEPS INVOLVED:

The overview of the whole process is as follows:

Step 1 :One query is picked at a time, it is negated and added to Knowledge base

Step 2: Resolution inference is applied using the method of Set of Sets to prove the query by inferencing a contradiction.

HOW TO EXECUTE:

- Make sure Resolution.py and input.txt are in the same directory before running the script file.
- When you run the Resolution.py file, it reads input.txt and performs resolution to prove the queries.
- The script generates output.txt which contains the True/False output for each query.
- True if a query can be inferred from the Knowledge base and False if it cannot be.

INPUT FILE: 1

6

F(Joe)

H(John)

~H(Alice)

~H(John)

G(Joe)

G(Tom)

14

$\sim F(x) \mid G(x)$

$\sim G(x) \mid H(x)$

$\sim H(x) \mid F(x)$

$\sim R(x) \mid H(x)$

$\sim A(x) \mid H(x)$

$\sim D(x,y) \mid \sim H(y)$

$\sim B(x,y) \mid \sim C(x,y) \mid A(x)$

$B(\text{John}, \text{Alice})$

$B(\text{John}, \text{Joe})$

$\sim D(x,y) \mid \sim Q(y) \mid C(x,y)$

$D(\text{John}, \text{Alice})$

$Q(\text{Joe})$

$D(\text{John}, \text{Joe})$

$R(\text{Tom})$

INPUT FILE 2:

2

$\text{Ancestor}(\text{Liz}, \text{Billy})$

$\text{Ancestor}(\text{Liz}, \text{Joe})$

6

$\text{Mother}(\text{Liz}, \text{Charley})$

$\text{Father}(\text{Charley}, \text{Billy})$

$\sim \text{Mother}(x,y) \mid \text{Parent}(x,y)$

$\sim \text{Father}(x,y) \mid \text{Parent}(x,y)$

$\sim \text{Parent}(x,y) \mid \text{Ancestor}(x,y)$

$\sim \text{Parent}(x,y) \mid \sim \text{Ancestor}(y,z) \mid \text{Ancestor}(x,z)$

1

```
kb.append(lines[i].strip("\n"))
```

```

        break

    input_file.close()

    return queries, kb

except IOError:

    fo = open(output_file, 'w')

    fo.write("File not found: {}".format(fin))

    fo.close()

    sys.exit()

def parseKB(kb):

    negativeKB = dict()

    positiveKB = dict()

    for item in kb:

        data = item.split('|')

        for i in data:

            i = i.replace(' ', '')

            if i[0] == '~':

                b = i[1:]

                b = b.partition("(")[0]

                try:

                    negativeKB[b].append(item)

                except KeyError:

                    negativeKB[b] = [item]

            else:

                i = i.partition("(")[0]

                try:

                    positiveKB[i].append(item)

                except KeyError:

                    positiveKB[i] = [item]

```

```
return negativeKB, positiveKB
```

```
def extract_constants(query):  
    variable = re.search(r'\((.*?)\)', query).group(1)  
    return variable
```

```
def checkSentence(kb):  
    if "|" in kb:  
        return False  
    const_list = re.search(r'\((.*?)\)', kb).group(1)  
    const = const_list.split(",")  
    for val in const:  
        if val[0].isupper():  
            continue  
        else:  
            return False  
    return True
```

```
def unification(query, left_over, positiveKB, negativeKB, can_simplify):  
    #print("In unification")  
    if query[0] != '~':  
        tomatch = query.partition("(")[0]  
        try:  
            value = negativeKB[tomatch]  
        except KeyError:  
            return False
```

```
for sentence in value:
    try:
        left_over_temp = left_over
        query_temp = query

        if sentence in can_simplify:
            ret1, l1 = remove(left_over_temp, sentence[1:])
            ret2 = 1
            l2 = ""
        else:
            ret1, l1 = remove(left_over_temp, query_temp)
            ret2, l2 = remove(sentence, "~" + query_temp)
        if ret1 == 0 or ret2 == 0:
            continue
        else:
            if l1 == "" and l2 != "":
                left_over_temp = l2
            elif l2 == "" and l1 != "":
                left_over_temp = l1
            elif l1 == "" and l2 == "":
                left_over_temp = ""
        else:
            left_over_temp = l2 + " | " + l1

        if left_over_temp == "":
            return True
        else:
            if "|" in left_over_temp:
                data = left_over_temp.split("|")
                for i in data:
                    i = i.replace(" ", "")
```

```

        if unification(i,left_over_temp,positiveKB,negativeKB,can_simplify):
            return True
        else:
            break
    else:
        if unification(left_over_temp,left_over_temp,positiveKB,negativeKB,can_simplify):
            return True
        else:
            continue
except RuntimeError as re:
    if re.args[0] == 'maximum recursion depth exceeded':
        return False

return False

else:
    tomatch = query.partition("(")[0]
    try:
        value = positiveKB[tomatch[1:]]
    except KeyError:
        return False
    for sentence in value:
        try:
            left_over_temp = left_over
            query_temp = query
            if sentence in can_simplify:
                ret_val1, l1 = remove(left_over_temp, "~" + sentence)
                ret_val2 = 1
                l2 = ""
            else:
                ret_val1, l1 = remove(left_over_temp, query_temp)

```

```

        ret_val2, l2 = remove(sentence, query_temp[1:])
if ret_val1 == 0 or ret_val2 == 0:
    continue
else:
    if l1 == "" and l2 != "":
        left_over_temp = l2
    elif l2 == "" and l1 != "":
        left_over_temp = l1
    elif l1 == "" and l2 == "":
        left_over_temp = ""
    else:
        left_over_temp = l2 + " | " + l1

if left_over_temp == "":
    return True
else:
    if "|" in left_over_temp:
        data = left_over_temp.split("|")
        for i in data:
            i = i.replace(" ", "")
            if unification(i, left_over_temp, positiveKB, negativeKB, can_simplifyfy):
                return True
        else:
            break
    else:
        if unification(left_over_temp, left_over_temp, positiveKB, negativeKB, can_simplifyfy):
            return True
        else:
            continue
except RuntimeError as re:
    if re.args[0] == 'maximum recursion depth exceeded':

```

```
        return False
    return False
```

```
def remove(k,query):
    __int, newq, news = substitution(k, query)
    if __int == 1:
        if newq in news:
            news1 = news.replace(newq, "")
        else:
            start = news.find(query.partition("(")[0])
            end = news.find(')', start)
            to_del = news[start:end + 1]
            news1 = news.replace(to_del, "")
        if " | | " in news1:
            news2 = news1.replace(" | | ", " | ")
            return 1,news2
        elif news1[:3] == " | ":
            news2 = news1[3:]
            return 1,news2
        elif news1[-3:] == " | ":
            news2 = news1[:-3]
            return 1,news2
        else:
            return 1, news1
    else:
        return 0,news
```

```
def substitution(sentence,query):
    predicate = query.partition("(")[0]
```

```

constant = extract_constants(query)
cons_list = constant.split(",")
count = 0

data = sentence.split(" | ")
flag = 0
for i in data:
    m = i.partition("(")[0]
    m = m.replace(' ', '')
    if m == predicate:
        __vars = re.search(r'\((.*?)\)', i).group(1)
        var_list = __vars.split(",")
        for j in var_list:
            if j[0].isupper() and cons_list[count][0].islower():
                query = test(cons_list[count], query, j)
                flag = 1
                count += 1
            elif j[0].islower() and cons_list[count][0].isupper():
                sentence = test(j, sentence, cons_list[count])
                flag = 1
                count += 1
            elif j[0].isupper() and cons_list[count][0].isupper():
                if j == cons_list[count]:
                    query = query
                    sentence = sentence
                    flag = 1
            else:
                flag = 0
                break
            count += 1

```

```

elif j[0].islower() and cons_list[count][0].islower():
    # print("both variables")
    if not (j == cons_list[count]):
        ## add code here

        sentence = test(j, sentence, cons_list[count])

        query = query

        flag = 1
    else:
        sentence = sentence

        query = query

        flag = 1

    count += 1

    if flag == 1:
        break

if flag == 0:
    return 0, query, sentence
else:
    return 1, query, sentence


def test(word, to_replace, with_replace):

    big_regex = re.compile(r'\b%s\b' % r'\b|\b'.join(map(re.escape, word)))

    a = big_regex.sub(with_replace, to_replace)

    return(a)


def main():

    output_file = "output.txt"

    fo = open(output_file, 'w')

    query_list, sentences = get_input()

    negativeKB, positiveKB = parseKB(sentences)

```

```

can_simplify = []
for a in sentences:
    if checkSentence(a):
        can_simplify.append(a)

for query in query_list:
    if query[0] == '~':
        new_query = query[1:]
        if unification(new_query, new_query, positiveKB, negativeKB, can_simplify):
            fo.write("TRUE" + "\n")
        else:
            fo.write("FALSE" + "\n")

    else:
        new_query = "~" + query
        if unification(new_query, new_query, positiveKB, negativeKB, can_simplify):
            fo.write("TRUE" + "\n")
        else:
            fo.write("FALSE" + "\n")
fo.close()

if __name__ == '__main__':
    main()

```

OUTPUT: 1

FALSE

TRUE

TRUE

FALSE

FALSE

TRUE

OUTPUT: 2

TRUE

FALSE

RESULT:

Thus the program for Resolution strategy was implemented and executed successfull

Ex.No:7 BUILDING NAÏVE BAYES MODEL

AIM:

To implement a program for Naïve Bayes model

NAÏVE BAYES CLASSIFIER ALGORITHM

- Naive Bayes is among one of the very simple and powerful algorithms for classification based on Bayes Theorem with an assumption of independence among the predictors.
- The Naive Bayes classifier assumes that the presence of a feature in a class is not related to any other feature.
- Naive Bayes is a classification algorithm for binary and multi-class classification problems.

Bayes Theorem

- Based on prior knowledge of conditions that may be related to an event, Bayes theorem describes the probability of the event
- conditional probability can be found this way
- Assume we have a Hypothesis(H) and evidence(E),
- According to Bayes theorem, the relationship between the probability of Hypothesis before getting the evidence represented as $P(H)$ and the probability of the hypothesis after getting the evidence represented as $P(H|E)$ is:

$$P(H|E) = P(E|H) * P(H) / P(E)$$

STEPS INVOLVE NAÏVE BAYES ALGORITHM

EXAMPLE: PIMA DIABETIC TEST

- The problem comprises 768 observations of medical details of the patients' records describes instantaneous measurement is taken from the patients such as they age, the number of times pregnant, blood workgroup.
- All the attributes are numeric and units vary from attribute to attribute. Each record has a class value that indicates whether the patients suffered on the set of diabetes within five years.
- The whole process can be brought down to five steps:

Step 1: Handling Data

Data is loaded from the CSV File and spread into training and tested assets.

Step 2: Summarizing the data

Summarise the properties in the training data set to calculate the probabilities and make predictions.

Step 3: Making a Prediction

A particular prediction is made using a summarise of the data set to make a single prediction

Step 4: Making all the Predictions

Generate prediction given a test data set and a summarise data set.

Step 4: Evaluate Accuracy:

Accuracy of the prediction model for the test data set as a percentage correct out of them all the predictions made.

Step 4: Trying all together

Finally, we tie to all steps together and form our own model of Naive Bayes Classifier.

PROGRAM:

```
# Importing library
import math
import random
import csv

# the categorical class names are changed to numeric data
# eg: yes and no encoded to 1 and 0
def encode_class(mydata):
    classes = []
    for i in range(len(mydata)):
        if mydata[i][-1] not in classes:
            classes.append(mydata[i][-1])
    for i in range(len(classes)):
        for j in range(len(mydata)):
            if mydata[j][-1] == classes[i]:
                mydata[j][-1] = i
    return mydata

# Splitting the data
def splitting(mydata, ratio):
    train_num = int(len(mydata) * ratio)
```

```

train = []
# initially testset will have all the dataset
test = list(mydata)
while len(train) < train_num:
    # index generated randomly from range 0
    # to length of testset
    index = random.randrange(len(test))
    # from testset, pop data rows and put it in train
    train.append(test.pop(index))
return train, test

```

```

# Group the data rows under each class yes or
# no in dictionary eg: dict[yes] and dict[no]

```

```

def groupUnderClass(mydata):
    dict = {}
    for i in range(len(mydata)):
        if (mydata[i][-1] not in dict):
            dict[mydata[i][-1]] = []
            dict[mydata[i][-1]].append(mydata[i])
    return dict

```

```

# Calculating Mean

```

```

def mean(numbers):
    return sum(numbers) / float(len(numbers))

```

```

# Calculating Standard Deviation

```

```

def std_dev(numbers):
    avg = mean(numbers)

```

```

variance = sum([pow(x - avg, 2) for x in numbers]) / float(len(numbers) - 1)
return math.sqrt(variance)

```

```

def MeanAndStdDev(mydata):
    info = [(mean(attribute), std_dev(attribute)) for attribute in zip(*mydata)]
    # eg: list = [ [a, b, c], [m, n, o], [x, y, z]]
    # here mean of 1st attribute =(a + m+x), mean of 2nd attribute = (b + n+y)/3
    # delete summaries of last class
    del info[-1]
    return info

```

find Mean and Standard Deviation under each class

```

def MeanAndStdDevForClass(mydata):
    info = {}
    dict = groupUnderClass(mydata)
    for classValue, instances in dict.items():
        info[classValue] = MeanAndStdDev(instances)
    return info

```

Calculate Gaussian Probability Density Function

```

def calculateGaussianProbability(x, mean, stdev):
    expo = math.exp(-(math.pow(x - mean, 2) / (2 * math.pow(stdev, 2))))
    return (1 / (math.sqrt(2 * math.pi) * stdev)) * expo

```

Calculate Class Probabilities

```

def calculateClassProbabilities(info, test):
    probabilities = {}
    for classValue, classSummaries in info.items():

```

```

probabilities[classValue] = 1
for i in range(len(classSummaries)):
    mean, std_dev = classSummaries[i]
    x = test[i]

    probabilities[classValue] *= calculateGaussianProbability(x, mean, std_dev)
return probabilities

```

Make prediction - highest probability is the prediction

```

def predict(info, test):
    probabilities = calculateClassProbabilities(info, test)
    bestLabel, bestProb = None, -1
    for classValue, probability in probabilities.items():
        if bestLabel is None or probability > bestProb:
            bestProb = probability
            bestLabel = classValue
    return bestLabel

```

returns predictions for a set of examples

```

def getPredictions(info, test):
    predictions = []
    for i in range(len(test)):
        result = predict(info, test[i])
        predictions.append(result)
    return predictions

```

Accuracy score

```

def accuracy_rate(test, predictions):
    correct = 0
    for i in range(len(test)):
        if test[i][-1] == predictions[i]:
            correct += 1

```

```
        return (correct / float(len(test))) * 100.0

# driver code

# add the data path in your system
filename = r'C:\python\diabet.csv'
# load the file and store it in mydata list
mydata = csv.reader(open(filename, "rt"))
mydata = list(mydata)
mydata = encode_class(mydata)
for i in range(len(mydata)):
    mydata[i] = [float(x) for x in mydata[i]]

# split ratio = 0.7
# 70% of data is training data and 30% is test data used for testing
ratio = 0.7
train_data, test_data = splitting(mydata, ratio)
print("Total number of examples are: ", len(mydata))
print("Out of these, training examples are: ", len(train_data))
print("Test examples are: ", len(test_data))

# prepare model
info = MeanAndStdDevForClass(train_data)

# test model
predictions = getPredictions(info, test_data)
accuracy = accuracy_rate(test_data, predictions)
print("Accuracy of your model is: ", accuracy)
```

OUTPUT:

Total number of examples are: 777

Out of these, training examples are: 543

Test examples are: 234

RESULT:

Thus the program for naïve Bayes model was implemented and executed successfully.

Ex.No:8 IMPLEMENT BAYESIAN NETWORKS AND PERFORM INFERENCE

AIM:

To perform inference on Bayesian network by implementing a Monty Hall Problem.

THE MONTY HALL PROBLEM

- Suppose you're on a game show, and you're given the choice of three doors: Behind one door is a car; behind the other two are goats.
 - You pick a door, say No.1, and the host, who knows what's behind the doors, opens another door, say No.3, which has a goat.
 - He then says to you, "Do you want to pick door No.2?"
 - There are three doors and 1 of the three doors contains the car, your desired prize, while 2 of the three doors contains the undesired prize (the goat).
 - If the contestant chooses to stick with their gut feeling and choose to never switch then the probability of winning is $1/3$ and their probability of losing is $2/3$.
After staying with your original choice: Probability (Winning) = $P(W) = 1/3$
After staying with your original choice: Probability (Losing) = $P(L) = 2/3$
- i. **Scenario #1 for always switching:**
1. You choose door #1
 2. The host opens door #3, because the prize is behind door #2, and asks if you want to switch.
 3. You decide to switch to door #2
 4. **YOU WIN!**
- ii. **Scenario #2 for always switching**
1. You choose door #2
 2. The host opens door #3 or door #1, because the prize is behind door #2, and asks if you want to switch.
 3. You decide to switch to door #1 or door #3
 4. **YOU LOSE.**
- iii. **Scenario #3 for always switching**
1. You choose door #3
 2. The host opens door #1, because the prize is behind door #2, and asks if you want to switch.
 3. You decide to switch to door #2
 4. **YOU WIN!**

iv.

PROGRAM:

```
#Play the game

def monty_hall_game(switch, num_tests):

    win_switch_cnt = 0

    win_no_switch_cnt = 0

    lose_switch_cnt = 0

    lose_no_switch_cnt = 0

    doors = [0,1,2] #Get the doors

    num_doors = len(doors) #Get the number of doors


    for i in range(0,num_tests):

        door_with_prize = random.randint(0, num_doors-1) #Randomly choose the door with the wanted prize

        host = door_with_prize #The host knows which door has the prize

        #The player chooses initially a random door that s/he believes has the prize

        player_choice = random.randint(0, num_doors-1)

        original_player_choice = player_choice

        shown_door = get_non_prize_door(host, num_doors, player_choice)

        if switch == True:

            player_choice = switch_function(shown_door,num_doors, player_choice)


        if player_choice == host and switch == False:

            #Then the player wins from not switching

            print('Player Wins (No switch) - The player chose door: ', player_choice,' Original choice: ',original_player_choice ,', Door with prize:', door_with_prize, ', Shown Door: ',shown_door )

            win_no_switch_cnt = win_no_switch_cnt + 1

        elif player_choice == host and switch == True:

            #Then the player wins from switching

            print('Player Wins (switch) - The player chose door: ', player_choice,' Original choice: ',original_player_choice ,', Door with prize:', door_with_prize, ', Shown Door: ',shown_door )

            win_switch_cnt = win_switch_cnt +1

        elif player_choice != host and switch == False:

            #The player lost from not switching

            print('Player Lost (No switch) - The player chose door: ', player_choice,' Original choice: ',original_player_choice ,', Door with prize:', door_with_prize, ', Shown Door: ',shown_door )

            lose_no_switch_cnt = lose_no_switch_cnt + 1
```

```
elif player_choice != host and switch == True:

    #The player lost from switching

    print('Player Lost (switch) - The player chose door: ', player_choice,' Original choice:
',original_player_choice , ', Door with prize:', door_with_prize, ', Shown Door: ',shown_door )

    lose_switch_cnt = lose_switch_cnt + 1

else:

    print('SOMETHING IS WRONG')

return win_no_switch_cnt,win_switch_cnt,lose_no_switch_cnt,lose_switch_cnt, num_tests

#Play the game

x = monty_hall_game(True, 10)

print('Win switch %: ', x[1]/ x[4])

print('Lose switch %: ', x[3]/ x[4])

print('Win No switch %: ', x[0]/ x[4])

print('Lose No switch %: ', x[2]/ x[4])
```

OUTPUT:

Player Wins (switch) - The player chose door: 2 Original choice: 0 , Door with prize: 2 , Shown Door: 1

Player Wins (switch) - The player chose door: 2 Original choice: 1 , Door with prize: 2 , Shown Door: 0

Player Lost (switch) - The player chose door: 0 Original choice: 2 , Door with prize: 2 , Shown Door: 1

Player Wins (switch) - The player chose door: 2 Original choice: 1 , Door with prize: 2 , Shown Door: 0

Player Wins (switch) - The player chose door: 2 Original choice: 1 , Door with prize: 2 , Shown Door: 0

Player Lost (switch) - The player chose door: 2 Original choice: 0 , Door with prize: 0 , Shown Door: 1

Player Lost (switch) - The player chose door: 0 Original choice: 1 , Door with prize: 1 , Shown Door: 2

Player Wins (switch) - The player chose door: 2 Original choice: 1 , Door with prize: 2 , Shown Door: 0

Player Lost (switch) - The player chose door: 0 Original choice: 1 , Door with prize: 1 , Shown Door: 2

Player Wins (switch) - The player chose door: 1 Original choice: 2 , Door with prize: 1 , Shown Door: 0

Win switch %: 0.6

Lose switch %: 0.4

Win No switch %: 0.0

Lose No switch %: 0.0

RESULT:

Thus the program to perform inference on Bayesian network by implementing a Monty Hall Problem was executed successfully.
